Discrete Optimal Planning

Discrete optimal planning

- π_K: a K-step plan which is a sequence $(u_1, u_2 \dots u_K)$ of K actions
- If x_I and π_K are given, then a seq of states $(x_1, x_2 \dots x_{K+1})$ can be derived using f.
- K is given. - X is finite

$x_1 = x_I$; $x_{i+1} = f(x_i, u_i)$

$F = K+1$.

- Deals with optimal planning problems
  - Rather than being satisfied with any sequence of actions that leads to the goal set.
- Three important extensions: $\Rightarrow$
  - Stage-wise execution of the plan
  - A cost function
  - A termination action

$L(\cdot)$: a cost fn

$$L(\pi_K) = \sum_{i=1}^{K} l(x_i, u_i) + l_F(x_F)$$

$l(x_i, u_i) \in \mathbb{R}$

$l_F(x_F) = 0$ if $x_F \in X_G$

$= \infty$ otherwise

* Try to understand why $L(\cdot)$ was introduced

- Remember: The aim is to find a plan that minimizes L.

Optimal fixed-length plans

- A naive solution: generate all length-K sequences of actions and select the sequence that produces the best cost.
  - But, this would cost $O(|U|^K)$ which is prohibitively high.
  - The dynamic programming principle – portions of optimal plans are themselves optimal – helps.
  - This leads to an iterative algorithm, called **value iteration**.
- Value iteration
  - Solves a vast collection of optimal planning problems, including those that involve variable-length plans, stochastic uncertainties, imperfect state measurements and many other complications.
  - Idea: compute optimal cost-to-go functions over the state space
  - **Backward value iteration**

Discrete fixed-length optimal planning – Formulation

- Optimal fixed-length plans
  - Naive approach: Generate all K-length sequences and select the one with best cost, but this takes $O(|U|^K)$ running time.
  - Dynamic programming helps (i) optimal substructure (ii) Overlapping sub-problems
    - Value iteration
      - Idea is to iteratively compute optimal cost-to-go (or cost-to-come) functions over the state space.

Backward value iteration

- Long optimal plans from shorter ones

For $i = 1$ to F, let G_i^* denote the cost that accumulate from stage i to F under optimal plan:

$$G_i^*(x_i) = \min_{u_i \dots u_K} \left\{ \sum_{j=i}^{K} l(x_j, u_j) + l_F(x_F) \right.$$

Discrete fixed-length optimal planning – Formulation

Boundary condn: $G_F^*(x_F) = l_F(x_F)$ — (1)

$\hookrightarrow F = K+1$

K passes over X ; Compute G_i^* from G_{i+1}^* where $i := K$ down to 1

For each $x_K \in X$, compute $G_K^*(x_K) = \min_{u_K} \{ l(x_K, u_K) + l_F(x_F) \}$ — (2)

$G_F^* \rightarrow G_K^*$ $\Rightarrow$ $= \min_{u_K} \{ l(x_K, u_K) + G_F^*(f(x_K, u_K)) \}$ (from (1))

$G_{i+1}^* \rightarrow G_i^*$

$$G_i^*(x_i) = \min_{u_i} \left\{ \min_{u_{i+1} \dots u_K} \left\{ l(x_i, u_i) + \sum_{j=i+1}^{K} l(x_j, u_j) + l_F(x_F) \right\} \right\}$$

$$G_i^*(x_i) = \min_{u_i} \{ l(x_i, u_i) + G_{i+1}^*(x_{i+1}) \}$$

Backward value iteration – example

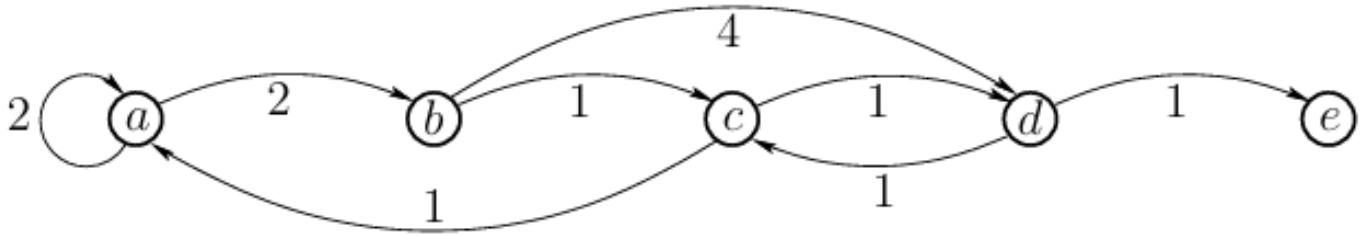


Figure 2.8: A five-state example. Each vertex represents a state, and each edge represents an input that can be applied to the state transition equation to change the state. The weights on the edges represent $l(x_k, u_k)$ (x_k is the originating vertex of the edge).

	a	b	c	d	e
G_5^*	∞	∞	∞	0	∞
G_4^*	∞	4	1	∞	∞
G_3^*	6	2	∞	2	∞
G_2^*	4	6	3	∞	∞
G_1^*	6	4	5	4	∞

Figure 2.9: The optimal cost-to-go functions computed by backward value iteration.

Backward value iteration – example

Backward value iteration – example

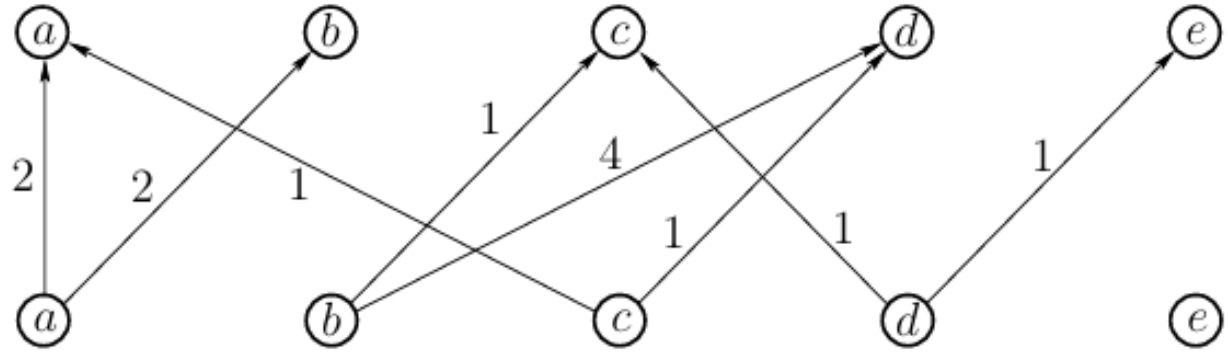


Figure 2.10: The possibilities for advancing forward one stage. This is obtained by making two copies of the states from Figure 2.8, one copy for the current state and one for the potential next state.